

****Comprehensive Guide to Modern App Development: Mobile, Desktop & Cross-Platform****

****1. Mobile App Development Ecosystem****

****1.1 Native vs Cross-Platform Development****

****Native Development:****

```java

// Android (Java/Kotlin) - MainActivity.kt

class MainActivity : AppCompatActivity() {

private lateinit var binding: ActivityMainBinding

override fun onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

binding = ActivityMainBinding.inflate(layoutInflater)

setContentView(binding.root)

binding.button.setOnClickListener {

val intent = Intent(this, DetailActivity::class.java)

startActivity(intent)

}

}

}

// iOS (Swift) - ViewController.swift

import UIKit

```

class ViewController: UIViewController {

 @IBOutlet weak var label: UILabel!

 @IBOutlet weak var button: UIButton!

 override func viewDidLoad() {
 super.viewDidLoad()

 button.addTarget(self, action: #selector(buttonTapped), for: .touchUpInside)
 }

 @objc func buttonTapped() {
 let detailVC = DetailViewController()
 navigationController?.pushViewController(detailVC, animated: true)
 }
}
...

```

**\*\*Cross-Platform with React Native:\*\***

```

```javascript
// App.js
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

const App = () => {
    const handlePress = () => {
        // Navigation logic
    };

    return (
        <View style={styles.container}>

```

```
<Text style={styles.title}>Welcome to My App</Text>

<TouchableOpacity style={styles.button} onPress={handlePress}>

  <Text style={styles.buttonText}>Get Started</Text>

</TouchableOpacity>

</View>

);

};
```

```
const styles = StyleSheet.create({

  container: {

    flex: 1,

    justifyContent: 'center',

    alignItems: 'center',

    backgroundColor: '#f5f5f5',

  },

  title: {

    fontSize: 24,

    fontWeight: 'bold',

    marginBottom: 20,

  },

  button: {

    backgroundColor: '#007AFF',

    paddingHorizontal: 20,

    paddingVertical: 10,

    borderRadius: 8,

  },

  buttonText: {

    color: 'white',

    fontSize: 16,
```

```
fontWeight: '600',  
},  
});
```

```
export default App;  
...
```

1.2 Flutter Development

```
```dart  
// main.dart
import 'package:flutter/material.dart';

void main() {
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'E-Commerce App',
 theme: ThemeData(
 primarySwatch: Colors.blue,
 visualDensity: VisualDensity.adaptivePlatformDensity,
),
 home: ProductListScreen(),
);
 }
}
```

```
}
```

```
class ProductListScreen extends StatefulWidget {
 @override
 _ProductListScreenState createState() => _ProductListScreenState();
}
```

```
class _ProductListScreenState extends State<ProductListScreen> {
 final List<Product> products = [
 Product(name: 'Laptop', price: 999.99, image: 'assets/laptop.jpg'),
 Product(name: 'Phone', price: 699.99, image: 'assets/phone.jpg'),
];
```

```
 @override
```

```
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Products'),
 actions: [
 IconButton(
 icon: Icon(Icons.shopping_cart),
 onPressed: () {
 Navigator.push(
 context,
 MaterialPageRoute(builder: (context) => CartScreen()),
);
 },
),
],
),
);
 }
```

```

),
 body: ListView.builder(
 itemCount: products.length,
 itemBuilder: (context, index) {
 return ProductCard(product: products[index]);
 },
),
);
}

class ProductCard extends StatelessWidget {
 final Product product;

 const ProductCard({Key? key, required this.product}) : super(key: key);

 @override
 Widget build(BuildContext context) {
 return Card(
 margin: EdgeInsets.all(8),
 child: ListTile(
 leading: Image.asset(product.image),
 title: Text(product.name),
 subtitle: Text("\${product.price}"),
 trailing: IconButton(
 icon: Icon(Icons.add_shopping_cart),
 onPressed: () {
 // Add to cart logic
 },
),
),
);
 }
}

```

```
),
),
);
}
}
```

```
class Product {
 final String name;
 final double price;
 final String image;

 Product({required this.name, required this.price, required this.image});
}
...

```

## \*\*2. Modern App Architecture Patterns\*\*

### \*\*2.1 MVVM (Model-View-ViewModel)\*\*

**\*\*Android with Jetpack Compose:\*\***

```kotlin

// Product.kt - Model

```
data class Product(  
    val id: String,  
    val name: String,  
    val price: Double,  
    val imageUrl: String,
```

```
    val isFavorite: Boolean = false
)
```

```
// ProductViewModel.kt - ViewModel
```

```
class ProductViewModel : ViewModel() {
    private val _products = MutableStateFlow<List<Product>>(emptyList())
    val products: StateFlow<List<Product>> = _products
```

```
    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading
```

```
    init {
        loadProducts()
    }
```

```
    private fun loadProducts() {
        viewModelScope.launch {
            _isLoading.value = true
            try {
                val products = productRepository.getProducts()
                _products.value = products
            } catch (e: Exception) {
                // Handle error
            } finally {
                _isLoading.value = false
            }
        }
    }
}
```

```

fun toggleFavorite(productId: String) {
    viewModelScope.launch {
        productRepository.toggleFavorite(productId)
        // Update local state
    }
}

```

// ProductScreen.kt - View

@Composable

```

fun ProductScreen(viewModel: ProductViewModel = hiltViewModel()) {
    val products by viewModel.products.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()

    if (isLoading) {
        LoadingIndicator()
    } else {
        LazyColumn {
            items(products) { product ->
                ProductItem(
                    product = product,
                    onFavoriteClick = { viewModel.toggleFavorite(product.id) }
                )
            }
        }
    }
}

```

@Composable

```
fun ProductItem(product: Product, onFavoriteClick: () -> Unit) {  
    Card(  
        elevation = 4.dp,  
        modifier = Modifier  
            .fillMaxWidth()  
            .padding(8.dp)  
    ){  
        Row(  
            modifier = Modifier.padding(16.dp),  
            verticalAlignment = Alignment.CenterVertically  
        ){  
            AsyncImage(  
                model = product.imageUrl,  
                contentDescription = product.name,  
                modifier = Modifier.size(64.dp)  
            )  
  
            Spacer(modifier = Modifier.width(16.dp))  
  
            Column(modifier = Modifier.weight(1f)) {  
                Text(  
                    text = product.name,  
                    fontWeight = FontWeight.Bold,  
                    fontSize = 16.sp  
                )  
                Text(  
                    text = "$${product.price}",  
                    color = Color.Gray,  
                    fontSize = 14.sp  
                )  
            }  
        }  
    }  
}
```

```

    )
  }

  IconButton(onClick = onFavoriteClick) {
    Icon(
      imageVector = if (product.isFavorite)
        Icons.Filled.Favorite else Icons.Outlined.Favorite,
      tint = if (product.isFavorite) Color.Red else Color.Gray,
      contentDescription = "Toggle favorite"
    )
  }
}
}
...

```

2.2 BLoC Pattern (Flutter)

```

``dart
// product_bloc.dart
import 'package:flutter_bloc/flutter_bloc.dart';

// Events
abstract class ProductEvent {}

class LoadProductsEvent extends ProductEvent {}

class ToggleFavoriteEvent extends ProductEvent {
  final String productId;
}

```

```
ToggleFavoriteEvent(this.productId);  
}
```

```
// States
```

```
abstract class ProductState {}
```

```
class ProductLoadingState extends ProductState {}
```

```
class ProductLoadedState extends ProductState {  
    final List<Product> products;
```

```
    ProductLoadedState(this.products);  
}
```

```
class ProductErrorState extends ProductState {  
    final String message;
```

```
    ProductErrorState(this.message);  
}
```

```
// BLoC
```

```
class ProductBloc extends Bloc<ProductEvent, ProductState> {  
    final ProductRepository repository;
```

```
    ProductBloc(this.repository) : super(ProductLoadingState()) {  
        on<LoadProductsEvent>(_onLoadProducts);  
        on<ToggleFavoriteEvent>(_onToggleFavorite);  
    }  
}
```

```

Future<void> _onLoadProducts(
    LoadProductsEvent event,
    Emitter<ProductState> emit
) async {
    emit(ProductLoadingState());

    try {
        final products = await repository.getProducts();
        emit(ProductLoadedState(products));
    } catch (e) {
        emit(ProductErrorState('Failed to load products'));
    }
}

```

```

Future<void> _onToggleFavorite(
    ToggleFavoriteEvent event,
    Emitter<ProductState> emit
) async {
    if (state is ProductLoadedState) {
        final currentState = state as ProductLoadedState;
        await repository.toggleFavorite(event.productId);

        final updatedProducts = currentState.products.map((product) {
            if (product.id == event.productId) {
                return product.copyWith(isFavorite: !product.isFavorite);
            }
            return product;
        }).toList();
    }
}

```

```

        emit(ProductLoadedState(updatedProducts));
    }
}
}

```

// Usage in UI

```

class ProductListScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (context) => ProductBloc(ProductRepository())..add(LoadProductsEvent()),
      child: Scaffold(
        appBar: AppBar(title: Text('Products')),
        body: BlocBuilder<ProductBloc, ProductState>(
          builder: (context, state) {
            if (state is ProductLoadingState) {
              return Center(child: CircularProgressIndicator());
            } else if (state is ProductErrorState) {
              return Center(child: Text(state.message));
            } else if (state is ProductLoadedState) {
              return ListView.builder(
                itemCount: state.products.length,
                itemBuilder: (context, index) {
                  final product = state.products[index];
                  return ProductItem(product: product);
                },
              );
            }
          }
        )
      )
    );
  }
}

```

```

        return Container();
    },
),
),
);
}
}
...

---
```

3. State Management Solutions

3.1 React Native with Redux Toolkit

```

```javascript
// store/index.js
import { configureStore, createSlice } from '@reduxjs/toolkit';

// Cart Slice
const cartSlice = createSlice({
 name: 'cart',
 initialState: {
 items: [],
 total: 0,
 },
 reducers: {
 addToCart: (state, action) => {
 const existingItem = state.items.find(
```

```

 item => item.id === action.payload.id
);

 if (existingItem) {
 existingItem.quantity += 1;
 } else {
 state.items.push({ ...action.payload, quantity: 1 });
 }

 state.total = calculateTotal(state.items);
},
removeFromCart: (state, action) => {
 state.items = state.items.filter(item => item.id !== action.payload);
 state.total = calculateTotal(state.items);
},
updateQuantity: (state, action) => {
 const { id, quantity } = action.payload;
 const item = state.items.find(item => item.id === id);

 if (item) {
 if (quantity <= 0) {
 state.items = state.items.filter(item => item.id !== id);
 } else {
 item.quantity = quantity;
 }
 }

 state.total = calculateTotal(state.items);
},

```

```
},
});
```

```
const calculateTotal = (items) => {
 return items.reduce((total, item) => total + (item.price * item.quantity), 0);
};
```

```
export const { addToCart, removeFromCart, updateQuantity } = cartSlice.actions;
```

```
// Store configuration
export const store = configureStore({
 reducer: {
 cart: cartSlice.reducer,
 },
});
```

```
// components/ProductList.js
import React from 'react';
import { View, Text, FlatList, TouchableOpacity } from 'react-native';
import { useDispatch, useSelector } from 'react-redux';
import { addToCart } from '../store';
```

```
const ProductList = () => {
 const dispatch = useDispatch();
 const cartItems = useSelector(state => state.cart.items);
```

```
 const products = [
 { id: '1', name: 'Laptop', price: 999.99 },
 { id: '2', name: 'Phone', price: 699.99 },
```

```
];
```

```
const handleAddToCart = (product) => {
 dispatch(addToCart(product));
};
```

```
const getItemQuantity = (productId) => {
 const item = cartItems.find(item => item.id === productId);
 return item ? item.quantity : 0;
};
```

```
return (
 <View style={styles.container}>
 <FlatList
 data={products}
 keyExtractor={item => item.id}
 renderItem={({ item }) => (
 <View style={styles.productCard}>
 <Text style={styles.productName}>{item.name}</Text>
 <Text style={styles.productPrice}>${item.price}</Text>
 <View style={styles.quantityContainer}>
 <Text>Quantity: {getItemQuantity(item.id)}</Text>
 <TouchableOpacity
 style={styles.addButton}
 onPress={() => handleAddToCart(item)}
 >
 <Text style={styles.buttonText}>Add to Cart</Text>
 </TouchableOpacity>
 </View>
 </View>
)
 >
 </View>
)
```

```
 </View>
 })
 />
 </View>
);
};
```

```
const styles = StyleSheet.create({
 container: {
 flex: 1,
 padding: 16,
 },
 productCard: {
 backgroundColor: 'white',
 padding: 16,
 marginBottom: 12,
 borderRadius: 8,
 shadowColor: '#000',
 shadowOffset: { width: 0, height: 2 },
 shadowOpacity: 0.1,
 shadowRadius: 4,
 elevation: 2,
 },
 productName: {
 fontSize: 18,
 fontWeight: 'bold',
 marginBottom: 8,
 },
 productPrice: {
```

```

 fontSize: 16,
 color: '#666',
 marginBottom: 12,
 },
 quantityContainer: {
 flexDirection: 'row',
 justifyContent: 'space-between',
 alignItems: 'center',
 },
 addButton: {
 backgroundColor: '#007AFF',
 paddingHorizontal: 16,
 paddingVertical: 8,
 borderRadius: 6,
 },
 buttonText: {
 color: 'white',
 fontWeight: '600',
 },
});

```

```

export default ProductList;
...

```

### \*\*3.2 SwiftUI with Combine\*\*

```

```swift
// ProductService.swift

import Foundation

```

```
import Combine
```

```
class ProductService: ObservableObject {
```

```
    @Published var products: [Product] = []
```

```
    @Published var isLoading = false
```

```
    @Published var error: String?
```

```
    private var cancellables = Set<AnyCancellable>()
```

```
    func loadProducts() {
```

```
        isLoading = true
```

```
        error = nil
```

```
        URLSession.shared.dataTaskPublisher(for: URL(string: "https://api.example.com/products")!)
```

```
            .map(\.data)
```

```
            .decode(type: [Product].self, decoder: JSONDecoder())
```

```
            .receive(on: DispatchQueue.main)
```

```
            .sink { [weak self] completion in
```

```
                self?.isLoading = false
```

```
                if case .failure(let error) = completion {
```

```
                    self?.error = error.localizedDescription
```

```
                }
```

```
            } receiveValue: { [weak self] products in
```

```
                self?.products = products
```

```
            }
```

```
            .store(in: &cancellables)
```

```
    }
```

```
}
```

```

// Product.swift

struct Product: Identifiable, Codable {

    let id: String

    let name: String

    let price: Double

    let imageUrl: String

    var isFavorite: Bool = false
}


// ProductListView.swift

import SwiftUI


struct ProductListView: View {

    @StateObject private var productService = ProductService()

    @State private var searchText = ""


    var filteredProducts: [Product] {

        if searchText.isEmpty {

            return productService.products

        } else {

            return productService.products.filter {

                $0.name.localizedCaseInsensitiveContains(searchText)

            }

        }

    }


    var body: some View {

        NavigationView {

            VStack {

```

```

        searchBar(text: $searchText)

        if productService.isLoading {
            progressView("Loading products...")
        } else if let error = productService.error {
            text("Error: \(error)")
            .foregroundColor(.red)
        } else {
            list(filteredProducts) { product in
                navigationLink(destination: ProductDetailView(product: product)) {
                    productRow(product: product)
                }
            }
            .listStyle(PlainListStyle())
        }
    }
    .navigationTitle("Products")
    .onAppear {
        productService.loadProducts()
    }
}
}
}

```

```

struct ProductRow: View {
    let product: Product
    @State private var isFavorite = false

    var body: some View {

```

```

HStack {
  AsyncImage(url: URL(string: product.imageUrl)) { image in
    image
      .resizable()
      .aspectRatio(contentMode: .fill)
  } placeholder: {
    Color.gray
  }
  .frame(width: 60, height: 60)
  .cornerRadius(8)

  VStack(alignment: .leading, spacing: 4) {
    Text(product.name)
      .font(.headline)
    Text("\$\\(product.price, specifier: "%.2f")")
      .font(.subheadline)
      .foregroundColor(.gray)
  }

  Spacer()

  Button(action: {
    isFavorite.toggle()
    // Update favorite status
  }) {
    Image(systemName: isFavorite ? "heart.fill" : "heart")
      .foregroundColor(isFavorite ? .red : .gray)
  }
}

```

```
        .padding(.vertical, 8)
    }
}
```

```
struct SearchBar: View {
    @Binding var text: String

    var body: some View {
        HStack {
            TextField("Search products...", text: $text)
                .padding(8)
                .background(Color(.systemGray6))
                .cornerRadius(8)

            if !text.isEmpty {
                Button(action: {
                    text = ""
                }) {
                    Image(systemName: "xmark.circle.fill")
                        .foregroundColor(.gray)
                }
            }
        }
        .padding(.horizontal)
    }
}

...

---
```

4. App Navigation Patterns

4.1 React Navigation (React Native)

```
``javascript
// navigation/AppNavigator.js
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';

// Screens
import HomeScreen from '../screens/HomeScreen';
import ProductDetailScreen from '../screens/ProductDetailScreen';
import CartScreen from '../screens/CartScreen';
import ProfileScreen from '../screens/ProfileScreen';
import SearchScreen from '../screens/SearchScreen';

const Stack = createNativeStackNavigator();
const Tab = createBottomTabNavigator();

// Tab Navigator
function HomeTabs() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, color, size }) => {
```

```

    let iconName;

    if (route.name === 'Home') {
      iconName = focused ? 'home' : 'home-outline';
    } else if (route.name === 'Search') {
      iconName = focused ? 'search' : 'search-outline';
    } else if (route.name === 'Cart') {
      iconName = focused ? 'cart' : 'cart-outline';
    } else if (route.name === 'Profile') {
      iconName = focused ? 'person' : 'person-outline';
    }

    return <Ionicons name={iconName} size={size} color={color} />;
  },
  tabBarActiveTintColor: '#007AFF',
  tabBarInactiveTintColor: 'gray',
  }}}
>
<Tab.Screen name="Home" component={HomeScreen} />
<Tab.Screen name="Search" component={SearchScreen} />
<Tab.Screen
  name="Cart"
  component={CartScreen}
  options={{ tabBarBadge: 3 }} // Dynamic badge
/>
<Tab.Screen name="Profile" component={ProfileScreen} />
</Tab.Navigator>
);
}

```

```
// Main App Navigator
function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="HomeTabs"
          component={HomeTabs}
          options={{ headerShown: false }}
        />
        <Stack.Screen
          name="ProductDetail"
          component={ProductDetailScreen}
          options={({ route }) => ({
            title: route.params.productName,
            headerBackTitle: 'Back'
          })}
        />
        <Stack.Screen
          name="Checkout"
          component={CheckoutScreen}
          options={{
            title: 'Checkout',
            presentation: 'modal'
          }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```
);  
}
```

```
export default AppNavigator;
```

```
// screens/HomeScreen.js
```

```
import React from 'react';
```

```
import { View, Text, TouchableOpacity } from 'react-native';
```

```
import { useNavigation } from '@react-navigation/native';
```

```
const HomeScreen = () => {
```

```
  const navigation = useNavigation();
```

```
  const handleProductPress = (product) => {
```

```
    navigation.navigate('ProductDetail', {
```

```
      productId: product.id,
```

```
      productName: product.name
```

```
    });
```

```
  };
```

```
  return (
```

```
    <View style={styles.container}>
```

```
      <Text style={styles.title}>Featured Products</Text>
```

```
      <TouchableOpacity
```

```
        style={styles.productCard}
```

```
        onPress={() => handleProductPress({
```

```
          id: '1',
```

```
          name: 'MacBook Pro'
```

```

    }}}
  >
  <Text>MacBook Pro</Text>
  <Text>$1999.99</Text>
</TouchableOpacity>

<TouchableOpacity
  style={styles.button}
  onPress={() => navigation.navigate('Checkout')}
>
  <Text style={styles.buttonText}>Proceed to Checkout</Text>
</TouchableOpacity>
</View>
);
};
...

```

4.2 Flutter Navigation

```

``dart
// main.dart - Navigation setup
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override

```

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    title: 'E-Commerce App',  
    theme: ThemeData(  
      primarySwatch: Colors.blue,  
      visualDensity: VisualDensity.adaptivePlatformDensity,  
    ),  
    initialRoute: '/',  
    routes: {  
      '/': (context) => HomeScreen(),  
      '/product': (context) => ProductDetailScreen(),  
      '/cart': (context) => CartScreen(),  
      '/checkout': (context) => CheckoutScreen(),  
    },  
    onGenerateRoute: (settings) {  
      if (settings.name == '/product') {  
        final args = settings.arguments as ProductDetailArguments;  
        return MaterialPageRoute(  
          builder: (context) {  
            return ProductDetailScreen(product: args.product);  
          },  
        );  
      }  
      return null;  
    },  
  );  
}
```

```
// navigation_service.dart

import 'package:flutter/material.dart';

class NavigationService {

  static final GlobalKey<NavigatorState> navigatorKey =
    GlobalKey<NavigatorState>();

  static Future<dynamic> navigateTo(String routeName, {Object? arguments}) {
    return navigatorKey.currentState!
      .pushNamed(routeName, arguments: arguments);
  }

  static Future<dynamic> navigateReplacement(String routeName,
    {Object? arguments}) {
    return navigatorKey.currentState!
      .pushReplacementNamed(routeName, arguments: arguments);
  }

  static void goBack() {
    return navigatorKey.currentState!.pop();
  }
}
```

```
// home_screen.dart

import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {

  @override

  Widget build(BuildContext context) {
```

```
return Scaffold(  
  appBar: AppBar(  
    title: Text('Products'),  
    actions: [  
      IconButton(  
        icon: Icon(Icons.shopping_cart),  
        onPressed: () {  
          Navigator.pushNamed(context, '/cart');  
        },  
      ),  
    ],  
  ),  
  body: ListView.builder(  
    itemCount: products.length,  
    itemBuilder: (context, index) {  
      final product = products[index];  
      return ProductCard(  
        product: product,  
        onTap: () {  
          Navigator.pushNamed(  
            context,  
            '/product',  
            arguments: ProductDetailArguments(product: product),  
          );  
        },  
      );  
    },  
  ),  
  bottomNavigationBar: BottomNavigationBar(  

```

```

        currentIndex: 0,
        items: [
            BottomNavigationBarItem(
                icon: Icon(Icons.home),
                label: 'Home',
            ),
            BottomNavigationBarItem(
                icon: Icon(Icons.search),
                label: 'Search',
            ),
            BottomNavigationBarItem(
                icon: Icon(Icons.person),
                label: 'Profile',
            ),
        ],
    ),
);
}
}

```

```

class ProductDetailArguments {
    final Product product;

    ProductDetailArguments({required this.product});
}
...

---
```

5. Data Persistence & Local Storage

5.1 SQLite with Room (Android)

```
``kotlin
```

```
// Product.kt
```

```
@Entity(tableName = "products")
```

```
data class Product(
```

```
    @PrimaryKey
```

```
    val id: String,
```

```
    val name: String,
```

```
    val price: Double,
```

```
    val imageUrl: String,
```

```
    val category: String,
```

```
    val isFavorite: Boolean = false,
```

```
    val lastUpdated: Long = System.currentTimeMillis()
```

```
)
```

```
// ProductDao.kt
```

```
@Dao
```

```
interface ProductDao {
```

```
    @Query("SELECT * FROM products")
```

```
    fun getAllProducts(): Flow<List<Product>>
```

```
    @Query("SELECT * FROM products WHERE id = :productId")
```

```
    fun getProductById(productId: String): Flow<Product?>
```

```
    @Query("SELECT * FROM products WHERE isFavorite = 1")
```

```
    fun getFavoriteProducts(): Flow<List<Product>>
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
suspend fun insertProduct(product: Product)
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
suspend fun insertAll(products: List<Product>)
```

```
@Update
```

```
suspend fun updateProduct(product: Product)
```

```
@Query("DELETE FROM products WHERE id = :productId")
```

```
suspend fun deleteProduct(productId: String)
```

```
@Query("UPDATE products SET isFavorite = :isFavorite WHERE id = :productId")
```

```
suspend fun updateFavoriteStatus(productId: String, isFavorite: Boolean)
```

```
}
```

```
// AppDatabase.kt
```

```
@Database(
```

```
    entities = [Product::class, CartItem::class],
```

```
    version = 1,
```

```
    exportSchema = false
```

```
)
```

```
abstract class AppDatabase : RoomDatabase() {
```

```
    abstract fun productDao(): ProductDao
```

```
    abstract fun cartDao(): CartDao
```

```
    companion object {
```

```
        @Volatile
```

```

private var INSTANCE: AppDatabase? = null

fun getInstance(context: Context): AppDatabase {
    return INSTANCE ?: synchronized(this) {
        val instance = Room.databaseBuilder(
            context.applicationContext,
            AppDatabase::class.java,
            "app_database"
        ).build()
        INSTANCE = instance
        instance
    }
}

// Repository.kt
class ProductRepository @Inject constructor(
    private val productDao: ProductDao,
    private val apiService: ApiService
) {
    fun getProducts(): Flow<List<Product>> = productDao.getAllProducts()

    suspend fun refreshProducts() {
        try {
            val products = apiService.getProducts()
            productDao.insertAll(products)
        } catch (e: Exception) {
            // Handle error
        }
    }
}

```

```

    }
}

suspend fun toggleFavorite(productId: String) {
    val product = productDao.getProductById(productId).first()
    product?.let {
        productDao.updateFavoriteStatus(productId, !it.isFavorite)
    }
}
}
...

```

5.2 Flutter with Hive

```

``dart
// hive_storage.dart
import 'package:hive/hive.dart';

class HiveStorage {
    static late Box<Product> productBox;
    static late Box<AppSettings> settingsBox;

    static Future<void> init() async {
        await Hive.initFlutter();

        // Register adapters
        Hive.registerAdapter(ProductAdapter());
        Hive.registerAdapter(AppSettingsAdapter());
    }
}

```

```

// Open boxes

productBox = await Hive.openBox<Product>('products');
settingsBox = await Hive.openBox<AppSettings>('settings');
}

// Product operations

static Future<void> saveProduct(Product product) async {
    await productBox.put(product.id, product);
}

static Future<void> saveProducts(List<Product> products) async {
    final Map<String, Product> productMap = {
        for (var product in products) product.id: product
    };
    await productBox.putAll(productMap);
}

static List<Product> getProducts() {
    return productBox.values.toList();
}

static Product? getProduct(String id) {
    return productBox.get(id);
}

static Future<void> deleteProduct(String id) async {
    await productBox.delete(id);
}

```

```
// Settings operations

static AppSettings getSettings() {
    return settingsBox.get('settings', defaultValue: AppSettings());
}

static Future<void> saveSettings(AppSettings settings) async {
    await settingsBox.put('settings', settings);
}
}
```

```
// product_model.dart
part 'product_model.g.dart';
```

```
@HiveType(typeId: 0)
class Product extends HiveObject {
    @HiveField(0)
    final String id;

    @HiveField(1)
    final String name;

    @HiveField(2)
    final double price;

    @HiveField(3)
    final String imageUrl;

    @HiveField(4)
    bool isFavorite;
```

```
Product({  
  required this.id,  
  required this.name,  
  required this.price,  
  required this.imageUrl,  
  this.isFavorite = false,  
});  
}
```

```
// app_settings.dart  
@HiveType(typeId: 1)  
class AppSettings extends HiveObject {  
  @HiveField(0)  
  bool isDarkMode;  
  
  @HiveField(1)  
  String language;  
  
  @HiveField(2)  
  bool notificationsEnabled;  
  
  AppSettings({  
    this.isDarkMode = false,  
    this.language = 'en',  
    this.notificationsEnabled = true,  
  });  
}  
...
```

6. API Integration & Networking

6.1 React Native with Axios

```
```\njavascript\n// api/client.js\nimport axios from 'axios';\nimport AsyncStorage from '@react-native-async-storage/async-storage';\n\nconst API_BASE_URL = 'https://api.example.com';\n\n// Create axios instance\nconst apiClient = axios.create({\n  baseURL: API_BASE_URL,\n  timeout: 10000,\n  headers: {\n    'Content-Type': 'application/json',\n  },\n});\n\n// Request interceptor\napiClient.interceptors.request.use(\n  async (config) => {\n    const token = await AsyncStorage.getItem('userToken');\n    if (token) {\n      config.headers.Authorization = `Bearer ${token}`;\n    }\n  },\n  (error) => Promise.reject(error)\n);
```

```
 }
 return config;
 },
 (error) => {
 return Promise.reject(error);
 }
);
```

```
// Response interceptor
apiClient.interceptors.response.use(
 (response) => response,
 (error) => {
 if (error.response?.status === 401) {
 // Handle unauthorized access
 // Redirect to login screen
 }
 return Promise.reject(error);
 }
);
```

```
export default apiClient;
```

```
// api/productApi.js
import apiClient from './client';
```

```
export const productApi = {
 // Get all products
 getProducts: async (params = {}) => {
 try {
```

```
 const response = await apiClient.get('/products', { params });
 return response.data;
 } catch (error) {
 throw new Error(error.response?.data?.message || 'Failed to fetch products');
 }
},
```

// Get single product

```
getProduct: async (productId) => {
 try {
 const response = await apiClient.get(`/products/${productId}`);
 return response.data;
 } catch (error) {
 throw new Error(error.response?.data?.message || 'Failed to fetch product');
 }
},
```

// Create product (admin)

```
createProduct: async (productData) => {
 try {
 const response = await apiClient.post('/products', productData);
 return response.data;
 } catch (error) {
 throw new Error(error.response?.data?.message || 'Failed to create product');
 }
},
```

// Update product

```
updateProduct: async (productId, updates) => {
```

```
 try {
 const response = await apiClient.put(`/products/${productId}`, updates);
 return response.data;
 } catch (error) {
 throw new Error(error.response?.data?.message || 'Failed to update product');
 }
 },

```

// Delete product

```
deleteProduct: async (productId) => {
 try {
 const response = await apiClient.delete(`/products/${productId}`);
 return response.data;
 } catch (error) {
 throw new Error(error.response?.data?.message || 'Failed to delete product');
 }
},
};
```

// hooks/useProducts.js

```
import { useState, useEffect } from 'react';
import { productApi } from '../api/productApi';
```

```
export const useProducts = (params = {}) => {
 const [products, setProducts] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);
```

```
 useEffect(() => {
```

```
 fetchProducts();
 }, [JSON.stringify(params)]); // Re-fetch when params change
```

```
const fetchProducts = async () => {
 try {
 setLoading(true);
 setError(null);
 const data = await productApi.getProducts(params);
 setProducts(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
};
```

```
const refetch = () => {
 fetchProducts();
};
```

```
return { products, loading, error, refetch };
};
```

// Usage in component

```
import React from 'react';
import { View, Text, FlatList, RefreshControl } from 'react-native';
import { useProducts } from '../hooks/useProducts';
```

```
const ProductList = () => {
```

```
const { products, loading, error, refetch } = useProducts({
 category: 'electronics',
 limit: 20
});

if (loading) {
 return <Text>Loading products...</Text>;
}

if (error) {
 return (
 <View>
 <Text>Error: {error}</Text>
 <Button title="Retry" onPress={refetch} />
 </View>
);
}

return (
 <FlatList
 data={products}
 keyExtractor={({item}) => item.id}
 renderItem={({ item }) => <ProductItem product={item} />}
 refreshControl={
 <RefreshControl refreshing={loading} onRefresh={refetch} />
 }
 />
);
};
```

...

---

## ## \*\*7. App Performance Optimization\*\*

### ### \*\*7.1 Image Optimization\*\*

```
```\javascript
// React Native Image Optimization
import React from 'react';
import { Image, View } from 'react-native';

const OptimizedImage = ({ source, style, ...props }) => {
  return (
    <View style={style}>
      <Image
        source={source}
        style={{ width: '100%', height: '100%' }}
        resizeMode="cover"
        progressiveRenderingEnabled={true}
        fadeDuration={300}
        {...props}
      />
    </View>
  );
};

// Image caching with react-native-fast-image
```

```
import FastImage from 'react-native-fast-image';
```

```
const CachedImage = ({ uri, style }) => {  
  return (  
    <FastImage  
      style={style}  
      source={{  
        uri: uri,  
        priority: FastImage.priority.normal,  
        cache: FastImage.cacheControl.immutable,  
      }}  
      resizeMode={FastImage.resizeMode.cover}  
    />  
  );  
};
```

```
// Flutter image optimization
```

```
import 'package:cached_network_image/cached_network_image.dart';
```

```
class OptimizedProductImage extends StatelessWidget {
```

```
  final String imageUrl;
```

```
  final double width;
```

```
  final double height;
```

```
  const OptimizedProductImage({
```

```
    Key? key,
```

```
    required this.imageUrl,
```

```
    required this.width,
```

```
    required this.height,
```

```
}) : super(key: key);
```